Practical List

1. Conversion of Infix expression to postfix expression using stack.
2. Conversion of Infix expression to prefix expression using stack.
3. Write a program to maintain multiple stacks in a single array.
4. Create a data structure **kQueues** that represents ‘k’ queues. Implementation of kQueues should use only one array, i.e., k queues should use the same array for storing elements. The following functions must be supported by kQueues.
    a. enqueue(int x, int qn): adds x to queue number ‘qn’ where qn is from 0 to k-1.
    b. dequeue(int qn): deletes an element from queue number ‘qn’ where qn is from 0 to k-1.
    c. displayq(int q): displays all the elements of the q specified.
    d. displayAll( ):displays the contents of all the queues.

5. Evaluation of Postfix expression using Stack
6. Evaluation of Prefix expression using Stack
7. Write a program to implement Queue using a circular array. The following operations need to be supported:
    a. enqueue
    b. dequeue
    c. isEmpty
    d. isFull
    e. front
    f. rear

8. Given a singly linked list, determine if it's a palindrome. Return 1 or 0 denoting if it's a palindrome or not, respectively. **For example**
    a. List 1 → 2 → 1 is a palindrome.

b. List 1 → 2 → 3 is not a palindrome.

9. Implement univariate Polynomial addition using linked lists. A univariate polynomial should be represented in the decreasing order of its exponent

10. Implement univariate Polynomial Multiplication using linked lists. A univariate polynomial should be represented in the decreasing order of its exponent .

11. Implement the following operations on doubly linked lists:

a. Insert (after position)
b. sort the list
c. display

12. Implement the following operations on doubly linked lists:

a. Insert (end)
b. concatenate two lists
c. display

13. Given a sorted linked list, delete all duplicates such that each element appears only once.

For example:

- Given 1→1 → 2, return 1 → 2
- Given 1 → 1 → 2 → 3 → 3, return 1 → 2 → 3

14. Given a linked list and a value x, partition it such that all nodes less than x come before nodes greater than or equal to x. You should preserve the original relative order of the nodes in each of the two partitions.

For example:

- Input: 1 → 5 → 3 → 2 → 4 → 2 and x = 3
- Output: 1 → 2 → 2 → 3 → 5 → 4

15. Write a program to construct a binary search tree and traverse it with all methods that use recursion.

16. Write a program to represent the given graph using adjacency matrix and implement Breadth-First Search Traversal for a given Graph.

17. Write a program to represent the given graph using adjacency linked list and implement Breadth-First Search Traversal for a given Graph.

18. Write a program to represent the given graph using adjacency matrix and implement Depth First Search Traversal for a given graph

19. Write a program to represent the given graph using adjacency linked list and implement Depth First Search Traversal for a given graph.

20. Write a program to implement a Hash Table using linear probing as the collision resolution strategy. The table should support the following operations:
    a. Insert
    b. Search
21. Write a program to implement a Hash Table using quadratic probing as the collision resolution strategy. The table should support the following operations:
    a. Insert
    b. Search
22. Implement a Min Heap tree and sort the elements. The following operations should be supported:
    a. heapify
    b. extractMin (Deleting the min element)
    c. heapsort

23. Add 2 non-negative numbers that have been given as a linked list

    Given two non-empty linked lists representing two non-negative integers. The most significant digit comes first and each of their nodes contains a single digit. Add the two numbers and return the

sum as a linked list. You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:

operand_1: 7 → 2 → 3 → 3

operand_2: 5 → 7 → 4

result: 7 → 8 → 0 → 7

24. Implement the Max Heap tree and sort the elements. The following operations should be supported:
    a. heapify
    b. extractMax (Deleting the max element)
    c. heapsort

25. Rotate a Linked List by ‘k’ places.

Example 1:

input: 1 → 2 → 3 → 4 → 5 , k = 1

output: 5 → 1 → 2 → 3 → 4

Example 2:

input: 1 → 2 → 3 → 4 → 5 , k = 2

output: 4 → 5 → 1 → 2 → 3

26. Swap nodes of a linked list in pairs

Example 1:

input: 1 → 2 → 3 → 4

output: 2 → 1 → 4 → 3

Example 2:

input: 1 → 2 → 3

output: 2 → 1 → 3

27. Implement recursive functions on Singly Linked List

a. Create b. Display c. Length d. Reverse

28. Create 2 singly linked lists in Ascending order and

a. merge b. Check if they are equal or not.

c. make a copy of the linked list.

29. On a Doubly Linked List perform the following operations

1. Create 2. Display 3. Insert an element 4. Delete an element

30. Implement stack using singly linked list

1. Push 2. Pop 3. Display topmost element 4. Display complete stack

31. Implement queue using singly linked list

1. enqueue 2. Dequeue 3. Display 4. Front 5. Rear

32. Create a binary tree (Recursive) and perform following operations.

1. InOrder traversal 2. Height of the tree

33. Create a binary tree and traverse the tree level by level.

34. Create a BST and print the output in Descending Order.

35. Create two hard coded trees with roots, root1 and root2 and

1. To check if they are equal or not.

2. To make mirror image by swapping every node with its sibling.

36. Create Binary tree and perform the following operations

1. Count terminal nodes 2. Count non terminal nodes 3.count total nodes

37. Create a binary tree and perform nonrecursive inorder traversal of it.

38. Given inorder and preorder traversal of a tree , construct a tree and perform postorder traversal (Recursive)

39. Create a binary tree and perform nonrecursive preorder traversal of it.

40. Create a binary tree and perform nonrecursive postorder traversal of it.